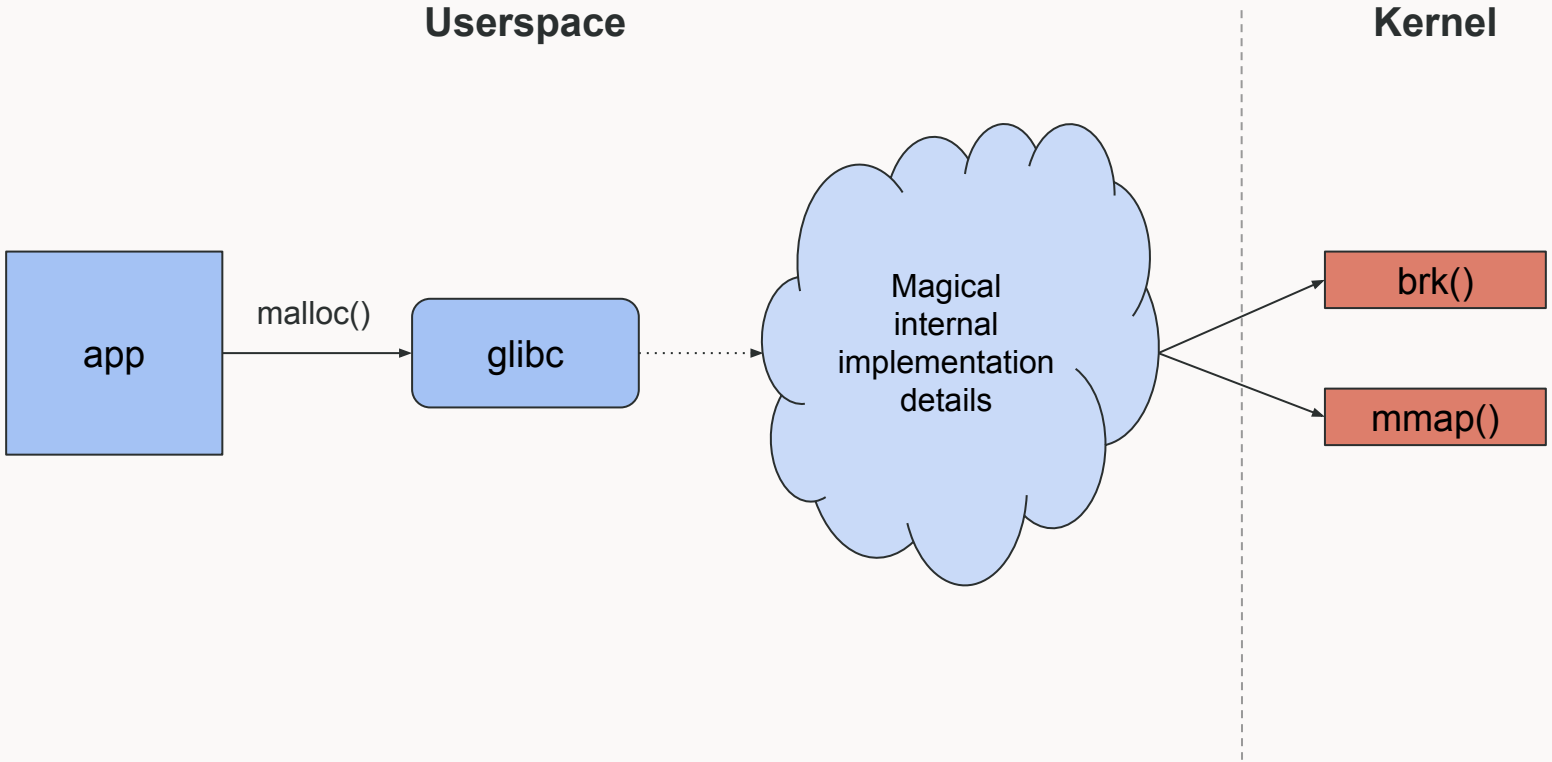
ORACLE

Where Does My Memory Come From?

Kernel Recipes 2025

Lorenzo Stoakes
Consulting Member of the Technical Staff
Oracle Cloud Infrastructure
September, 2025

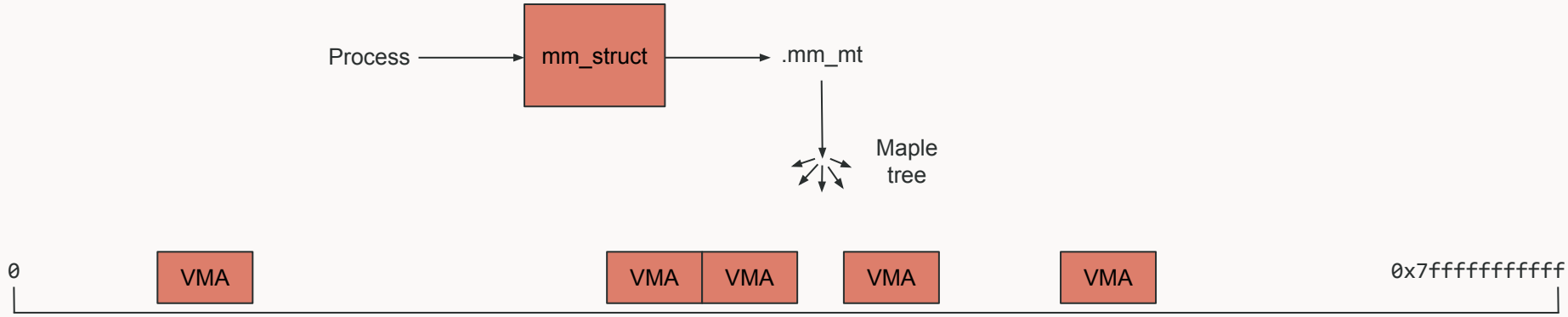
Where Does My Memory Come From? Take 1



Copyright © 2025, Oracle and/or its affiliates

Well no, not really

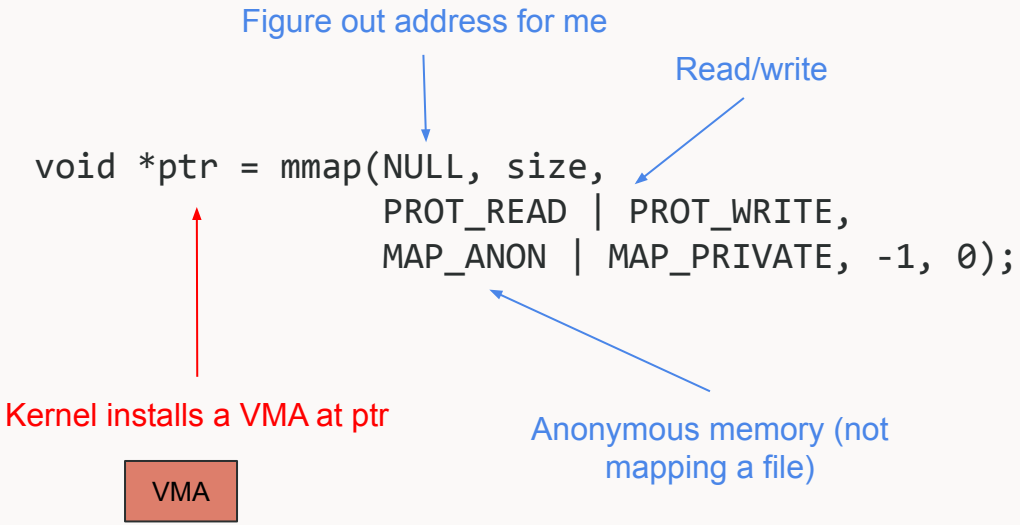
Not actually allocating anything



Copyright © 2025, Oracle and/or its affiliates

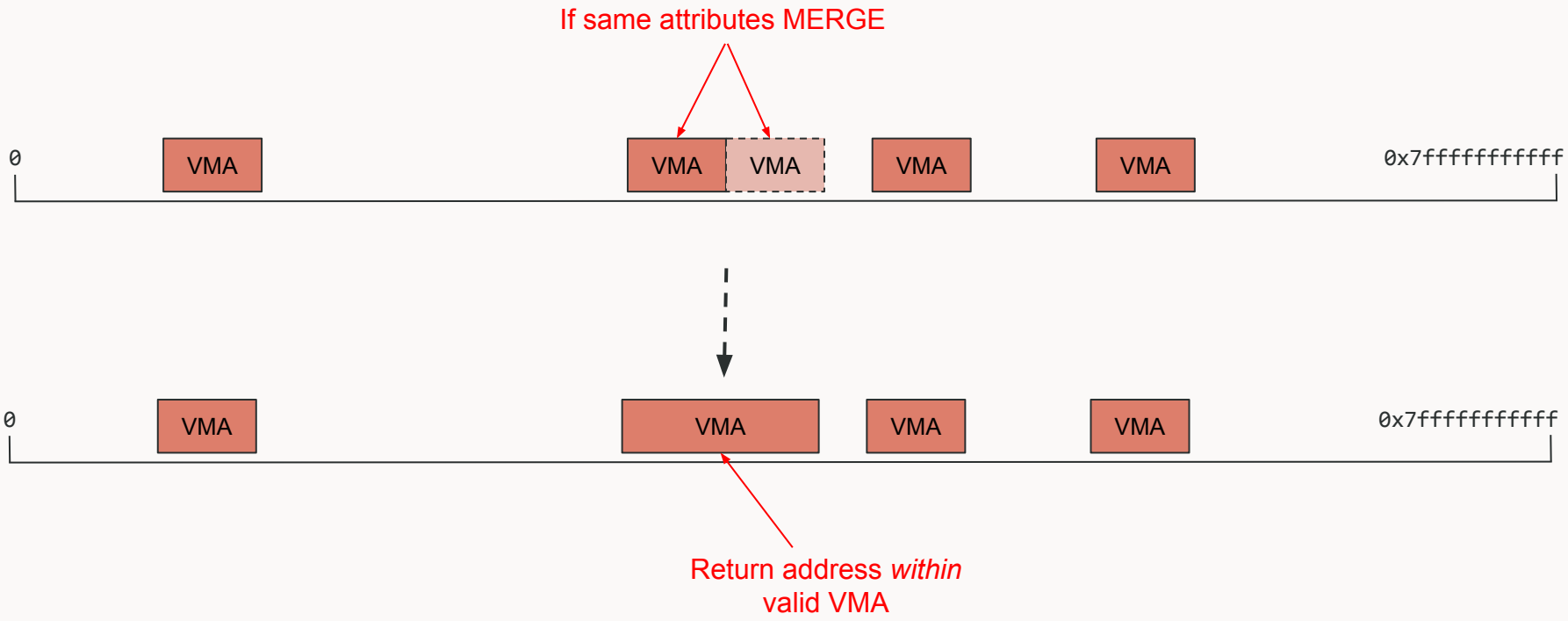
OK, So How Does Userland Ask For a VMA?

The modern way is mmap():



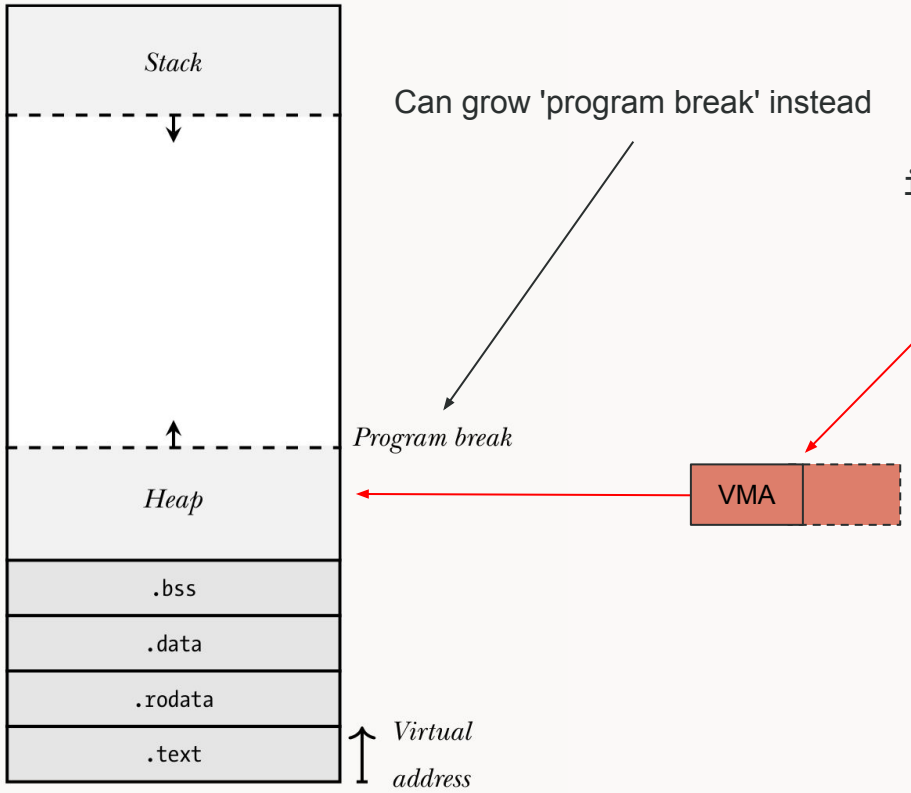
Copyright © 2025, Oracle and/or its affiliates

Well OK, Not Quite



Copyright © 2025, Oracle and/or its affiliates

Also, brk()



Can grow 'program break' instead

```
if (!brk(new_brk_addr))
        ... allocation succeeded ...
```

VMA

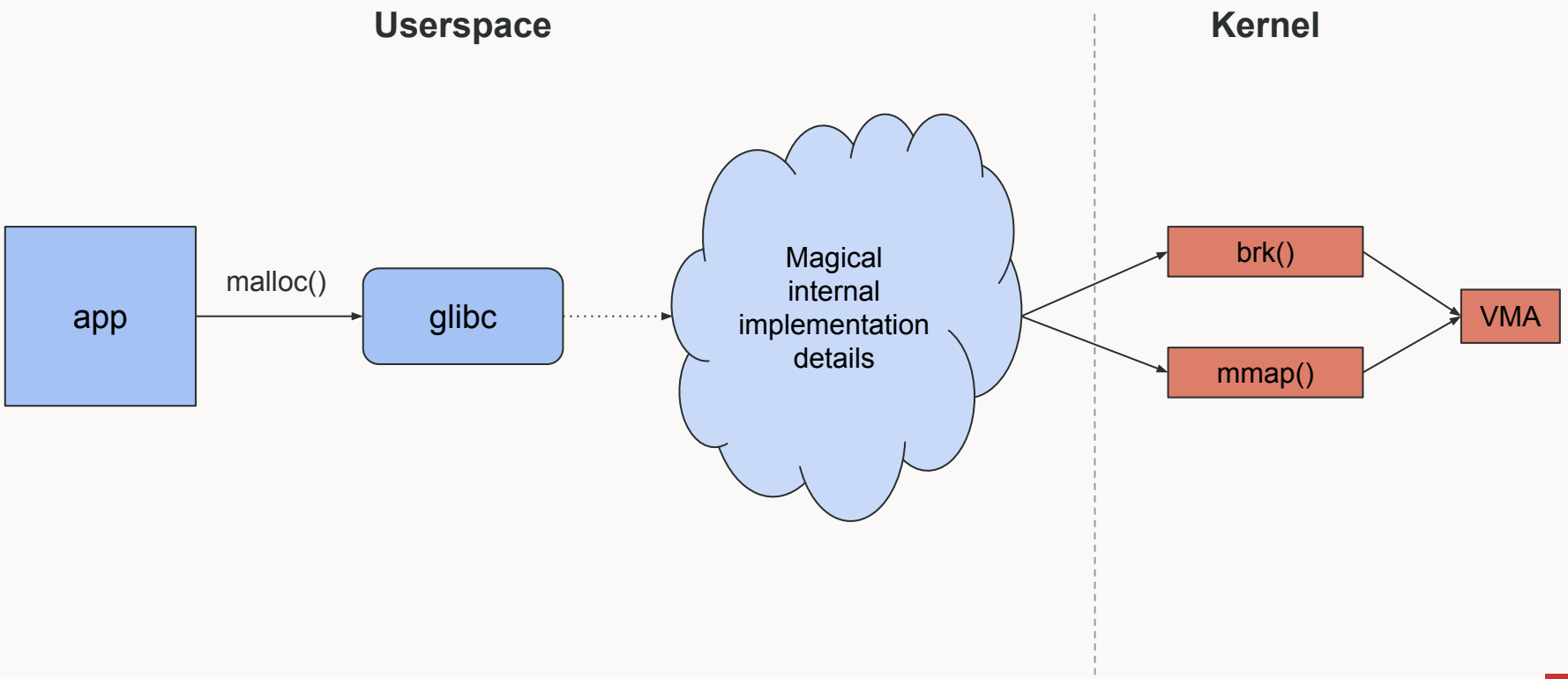
If **system call** succeeds, returns END of valid range, i.e. the 'break'.

Otherwise returns old break.

libc function just returns 0 on success, -1 on failure. Can use **sbrk()** function to get current brk.

Copyright © 2025, Oracle and/or its affiliates

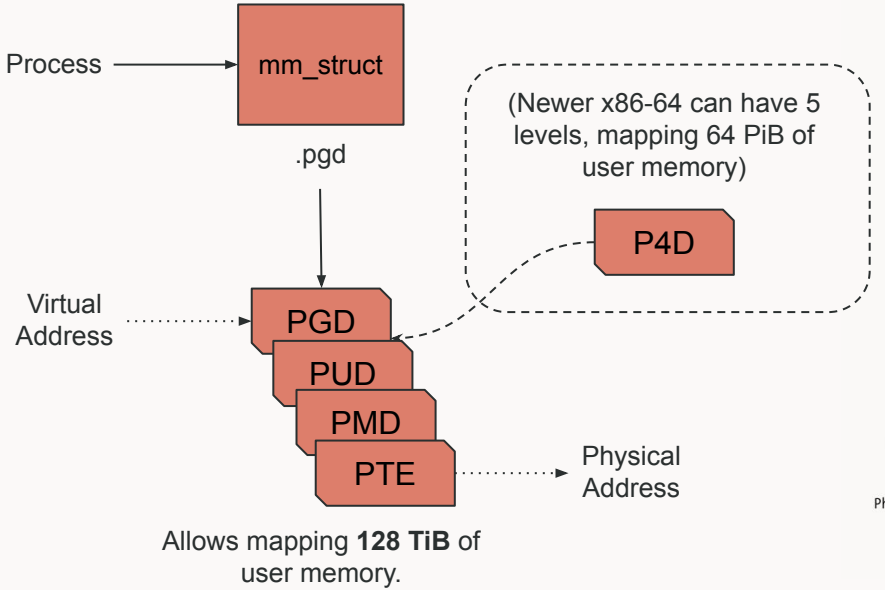
Where Does My Memory Come From? Take 2

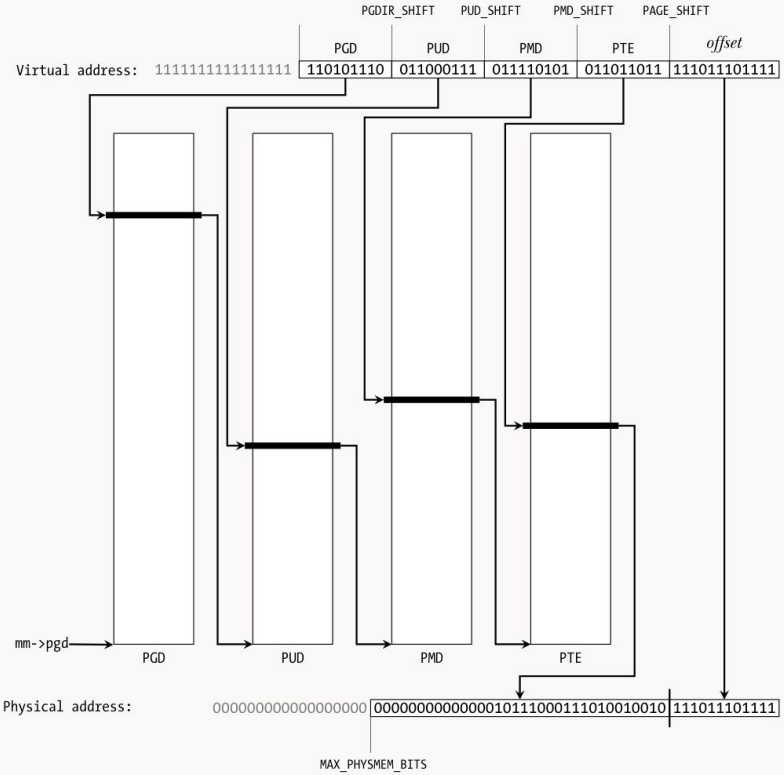


Copyright © 2025, Oracle and/or its affiliates

Well... Not Quite

In modern, **sane** hardware, memory is **virtual** and uses **page tables** to 'map' memory.





Copyright © 2025, Oracle and/or its affiliates

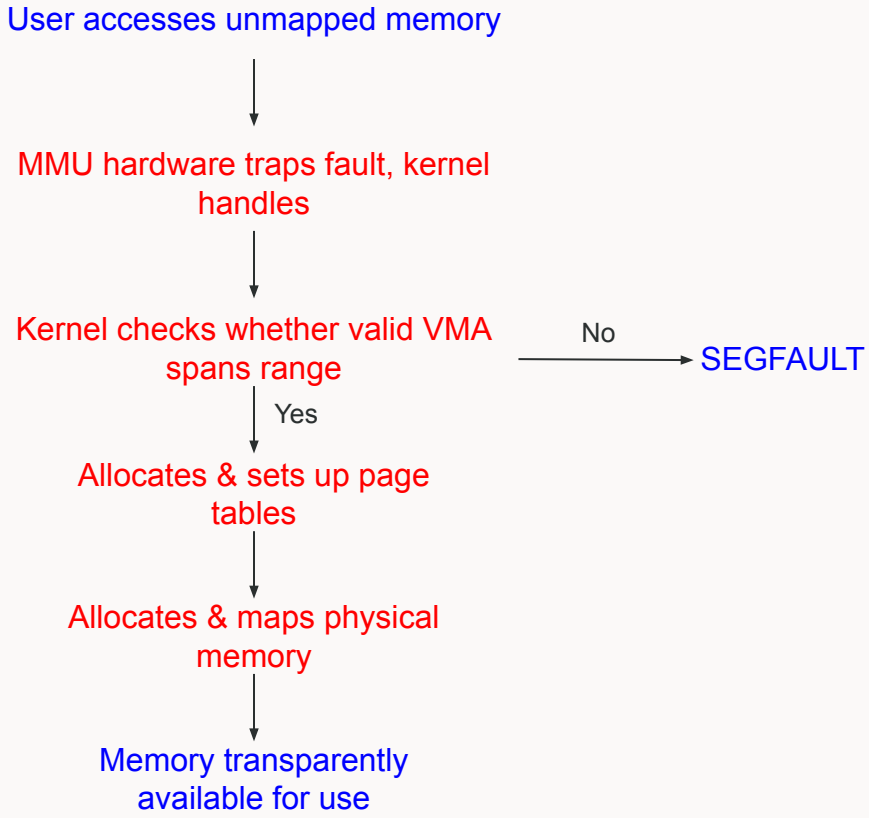
Page Faults

When userland tries to access memory that doesn't have page tables set up:

1. The CPU detects this and triggers a **page fault**.

2. The kernel **handles** the page fault.

3. If address is valid, the kernel transparently **allocates** memory and sets up the **mapping**.



Copyright © 2025, Oracle and/or its affiliates

Physical Allocation

Check freelists based on 2^n page size, where n is **'order'**.

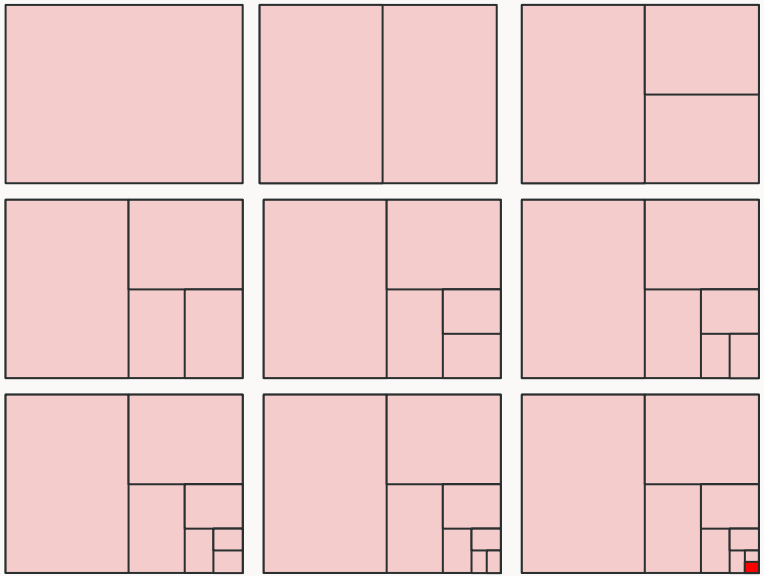
If required size not available, keep dividing next available block size by two until we have what we need.

The 'other half' of the divided blocks is each block's '**buddy**', e.g. `page_pfn ^ (1 << order)`.

When freeing, '**coalesce**' blocks by combining freed block and buddy, if that buddy is already free.

Invented in 1963 by **Harry Markowitz**, known to be highly efficient (& helpfully - simple!)

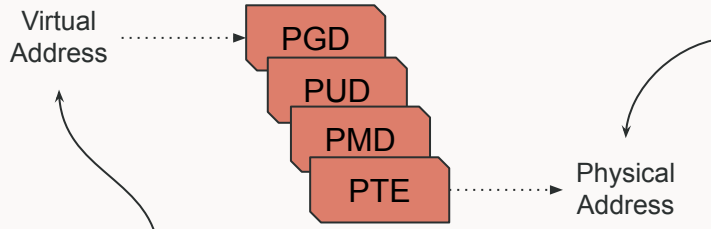
Described very well in The Art of Computer Programming by **Donald Knuth**.



Copyright © 2025, Oracle and/or its affiliates

Folios

Page tables map virtual addresses to physical ones for userland memory:



Metadata: **VMA**

We store metadata describing virtual mappings using **VMAs**.

We store metadata describing virtual mappings using **Folios**.

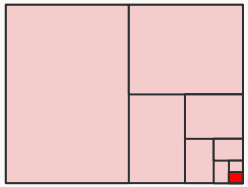
Metadata: **Folio**

Folios describe physical memory that is:

Physically contiguous.
Virtually contiguous.
Mappable into userland.

The physical memory is **aligned** but can be mapped arbitrarily in virtual memory.

Folios are, of course, always page-aligned.



Folios can be of any (valid) **order** and use pages allocated by the **physical allocator**.

Copyright © 2025, Oracle and/or its affiliates

Anonymous vs. File-backed

The kernel makes an important distinction between '**anonymous**' memory and '**file-backed**' mappings.

File-backed mappings map a file, **anonymous** mappings do not.





From the kernel perspective, **VMAs** are anonymous if they do **not** map a file, but **folios** are anonymous if they do not live in the page cache.

More useful point of view - **swap-backed** vs. **file-backed***.

(* edge cases: MADV_FREE sets anon folios non-swap-backed, shmem)

Folios belonging to file-backed mappings **live in the page cache**. This is, as it sounds, a cache of... err... folios for file-backed memory.

Page cache folio ranges are tied (ultimately) to **inodes**. **Can look them up easily**, so when mapping a file we can very quickly do so**.**

Anonymous memory simply consists of arbitrarily allocated memory. **Cannot look them up easily.**

Importantly: we **reclaim** memory based on whether the **folio** is anonymous or not.

Generally: when referring to anonymous **memory** we mean the **folios**. When referring to anonymous **mappings** we mean the **VMAs**.

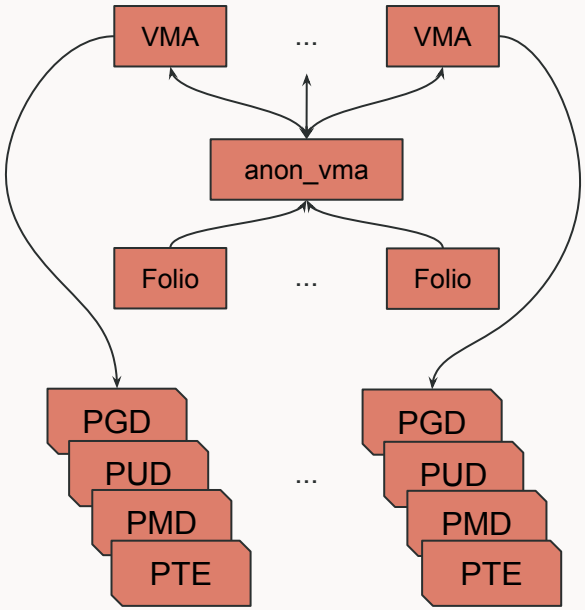
Of course this is mm so we make life difficult for ourselves:

- **shmem** (tmpfs et al.) mappings have **file-backed VMAs** and **folios**, are in the **page cache** but are **swap-backed** + reclaimed as anon.
- A user can map **file-backed** memory **MAP_PRIVATE**, which means it maps **file-backed** folios up until the mapping is written to, at which point we copy them into **anonymous** folios.

Copyright © 2025, Oracle and/or its affiliates

Reverse Mapping

Anonymous



We deal with physical memory at **folio** level, but need to look up **VMAs** too (e.g. for unmapping reclaimed memory or truncated files). We use the **reverse mapping** for this.

The reverse mapping is established between VMAs and a **mapping** object.

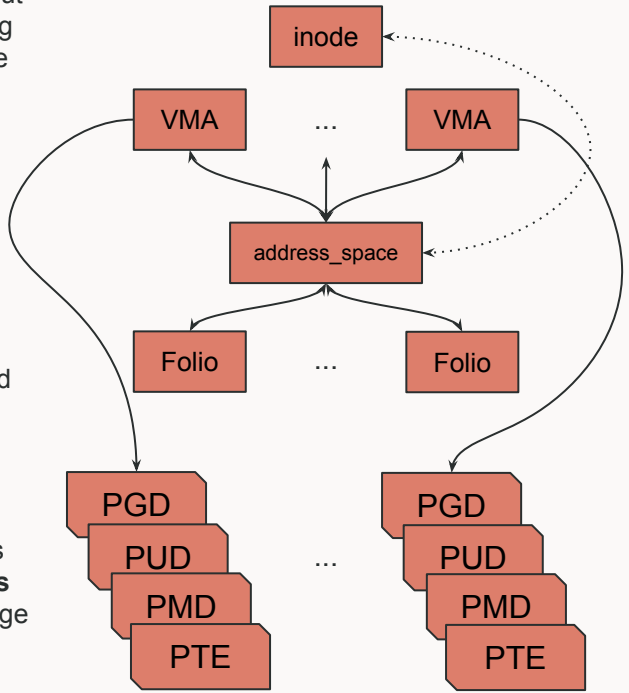
Each 'related' VMA is tracked from mapping via an interval tree.

Anonymous folios map to an **anon_vma**, and file-backed folios to an **address_space.**

Folios track their offset in the mapping.

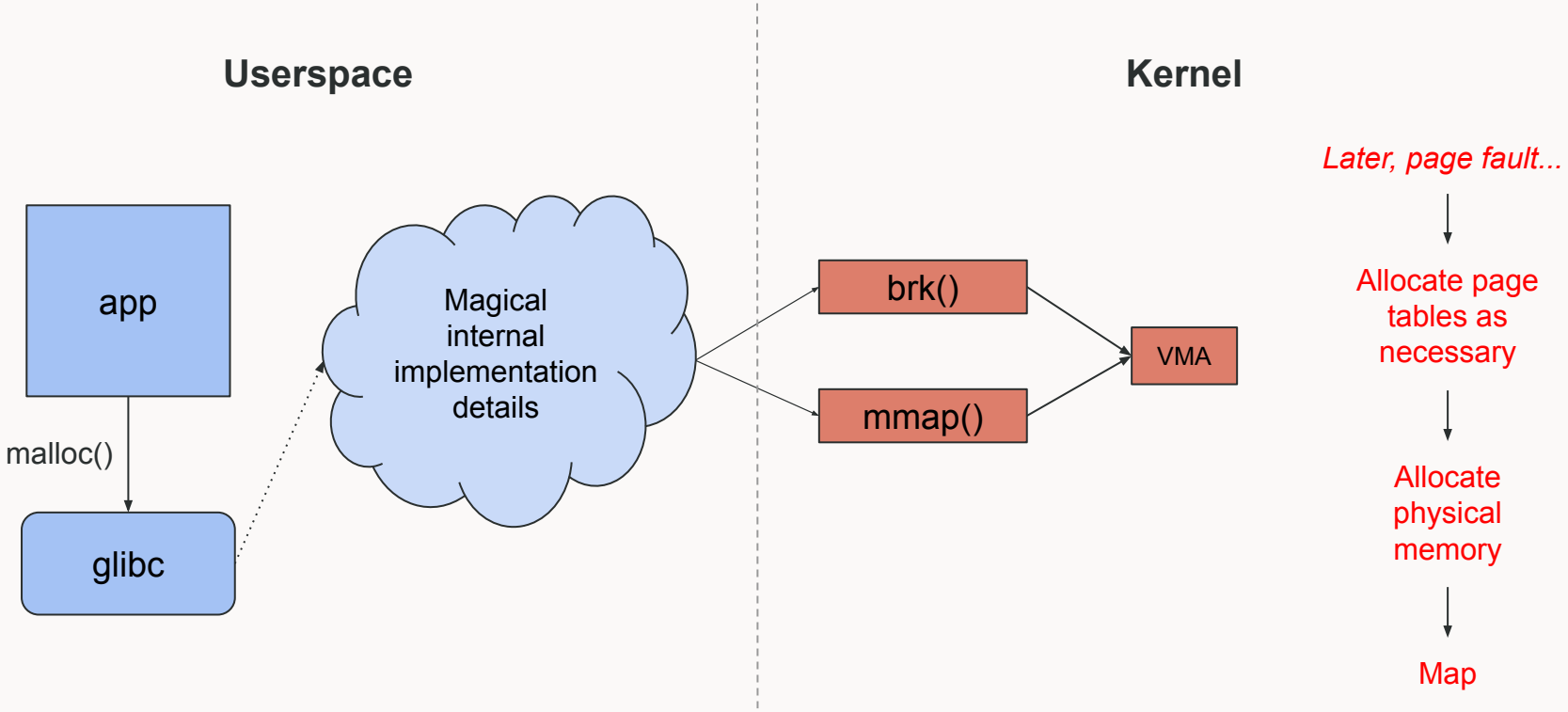
A typical reverse mapping operation moves from the **folio** via the **mapping** to the **VMAs** and from there ascertains virtual address range which are used to walk **page tables**.

File-backed



Copyright © 2025, Oracle and/or its affiliates

Where Does My Memory Come From? Take 3



Copyright © 2025, Oracle and/or its affiliates

Closer... But the Memory Might Not Stick Around

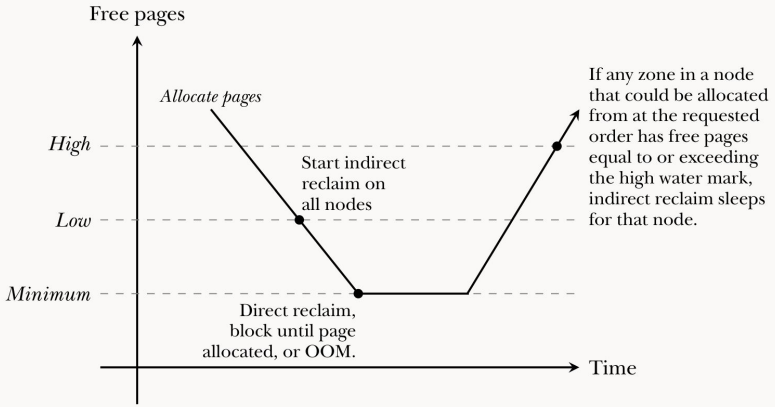
Under memory pressure, or madvise() we **reclaim** the least most recently used memory.	When enough PTEs are faulted in, we can replace it with a **huge page** via **khugepaged**.	The user might force a **zap** of memory via madvise(MADV_DONTNEED).	When we can't allocate higher order pages we need to perform **compaction**.	Of course, a user might simply **munmap()** memory or free via **brk()**.	User uses the **cgroup** cpuset mechanism to constrain tasks to node, causing migrate.
DAMON may trigger migration of memory via DAMOS_MIGRATE_HOT/COLD.	Files mapped into memory might be deleted, which **truncates** mappings.	A user can fault in a range and force conversion to huge page via **MADV_COLLAPSE.**	**NUMA balancing** clears memory mappings which might migrate.	User invokes **NUMA** mbind(), migrate_pages(), move_pages() system calls, causing migrate.	**mremap()** moves memory, unmapping existing mapping.
userfaultfd memory move using UFFDIO_MOVE, unmapping original memory.	Long-term **GUP pin** migrates now-unmovable pages away from movable ones.	**Copy-on-Write** anon/ MAP_PRIVATE file-backed write-fault.	Driver might **migrate** user mapped pages to device memory (e.g. GPU to VRAM).	madvise(**MADV_DONTFORK**) mappings don't exist in child post-fork.	And more...

Copyright © 2025, Oracle and/or its affiliates

Reclaim 1

Reclaim is the kernel's means of **freeing up memory** under **memory pressure.** Approximates a **Least Recently Used (LRU)** algorithm.

What is memory pressure?

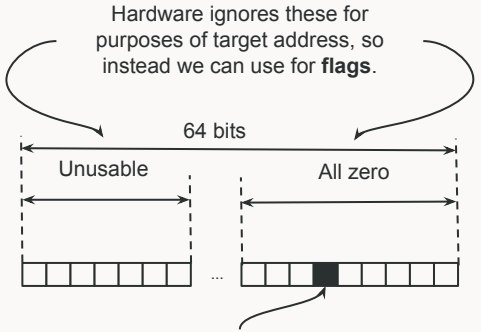


Memory pressure simply means that **free** memory is getting low.

Use **watermarks** to figure out what to do.

How to figure out if recently used?

PTE entry



Accessed flag is set high every time the page table is accessed.

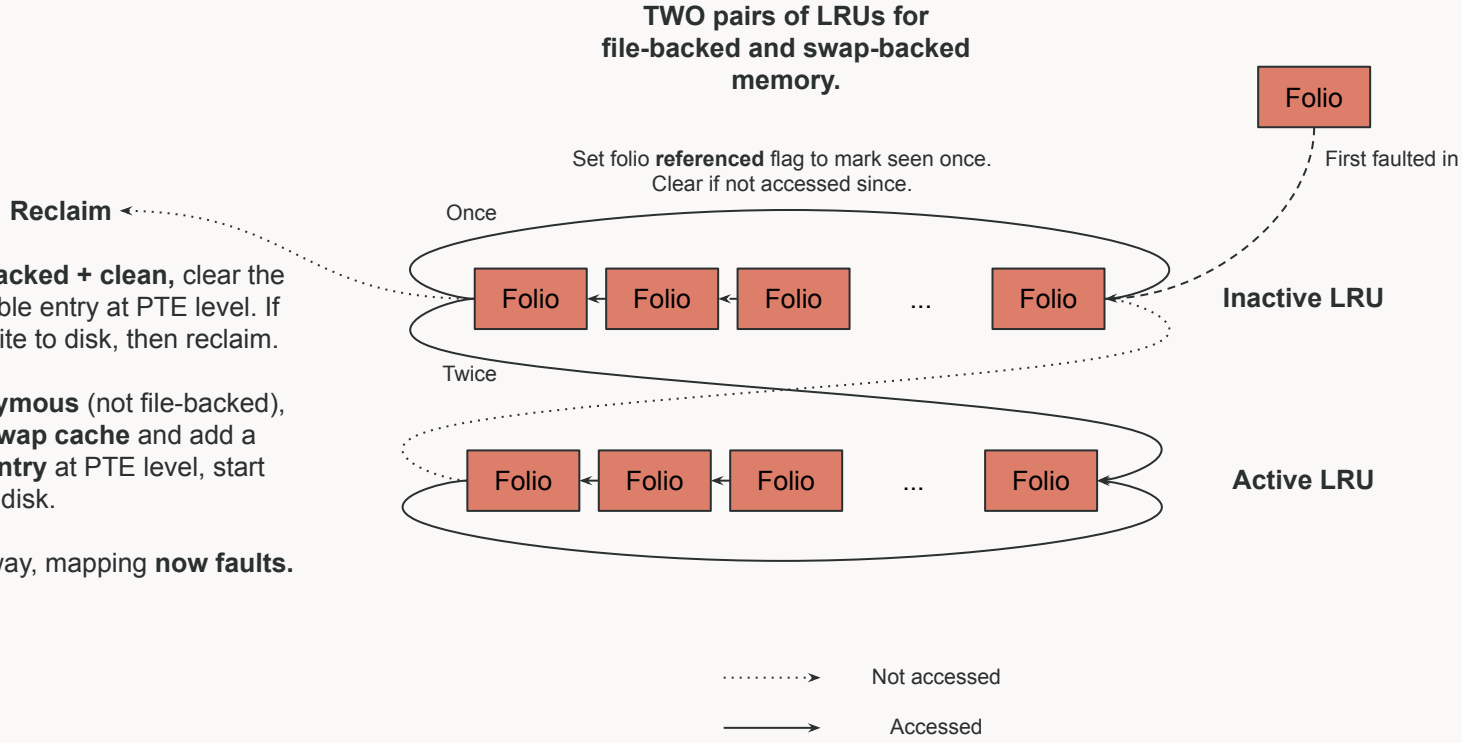
The kernel can **clear** it so it knows:

"This page was accessed since I lasted cleared this flag."

Copyright © 2025, Oracle and/or its affiliates

Reclaim 2

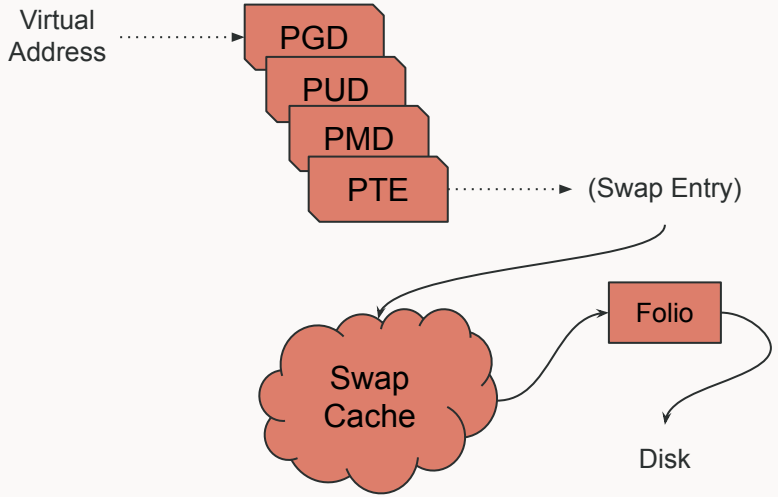
1. If **file-backed + clean,** clear the page table entry at PTE level. If dirty, write to disk, then reclaim.
2. If **anonymous** (not file-backed), put in **swap cache** and add a **swap entry** at PTE level, start write to disk.
3. Either way, mapping **now faults.**



Copyright © 2025, Oracle and/or its affiliates

Swap

Swap OUT

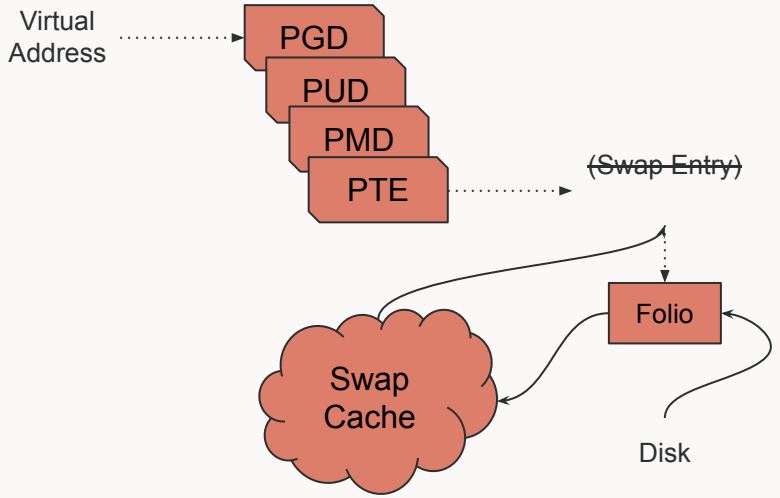


1. Put folio in **swap cache**, replace PTE entry with **swap entry** *for every mapping*. Mapping will now **fault.**

2. Start async **writeback** of folio to swap.

3. When writeback done, **free** folio from swap cache.

Swap IN



1. Check if already in swap cache, if not, start **async read** of folio to swap cache.

2. When done, replace **swap entry** with folio. No longer faults.

3. If no swap entries left, **remove** from swap cache.

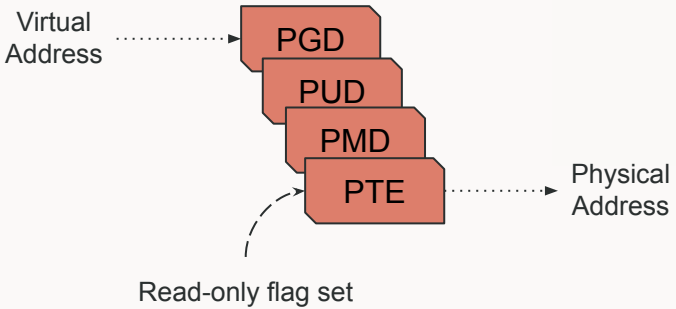
Copyright © 2025, Oracle and/or its affiliates | Confidential: Internal/Restricted/Highly Restricted

Forking & Copy-on-Write (CoW)

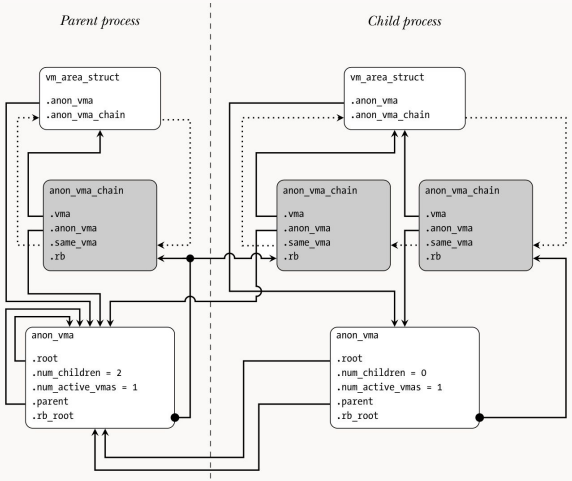
Anonymous memory seems to be simple, but **forking** makes it not so simple.

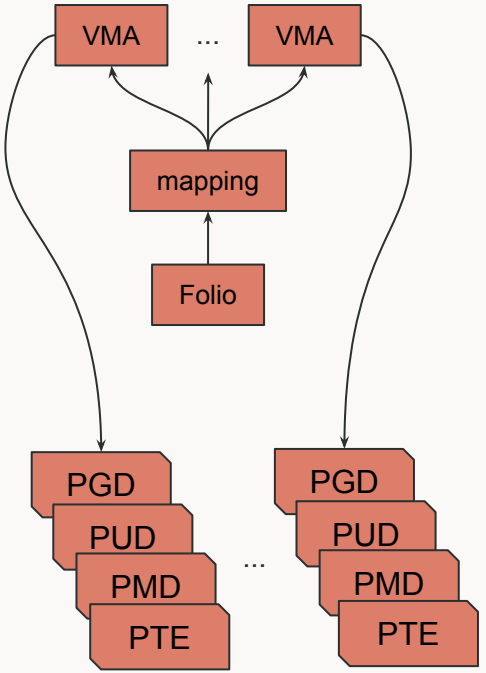
For efficiency, when we fork, we copy page tables, marking the 'leaf' entries (PTE, or if huge higher) entries **read-only**.

If somebody writes, then we **fault** - if they are the **exclusive** mapper it is just marked read/write otherwise we **copy.**



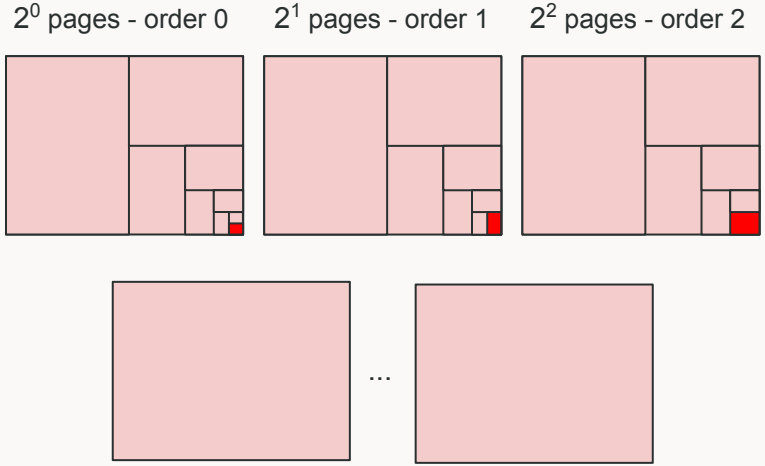
Things get complicated fast, as one process can fork another which can fork another, etc. etc.





Copyright © 2025, Oracle and/or its affiliates

Compaction/Migration 1



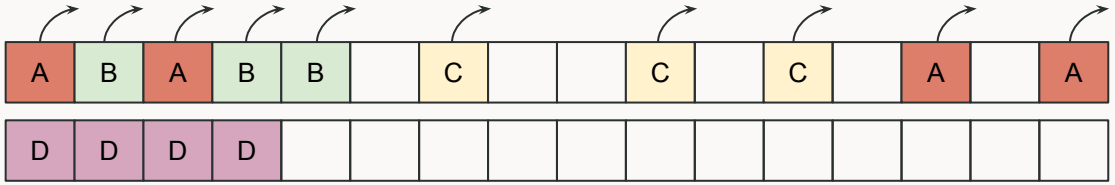
When we physically allocate memory, we allocate sizes based on **physically contiguous** pages.

To reduce **fragmentation** between userland/kernel we place userland (movable) memory in **pageblocks** (typically order-9).

The physical allocator will **coalesce** smaller pages into larger ones when freed. But sometimes they get **fragmented.**

Isolate and **migrate** folios* to other pageblocks

D needs order 2 (4 pages):



**folios comprise of the order-N physical page provided by the physical allocator.*

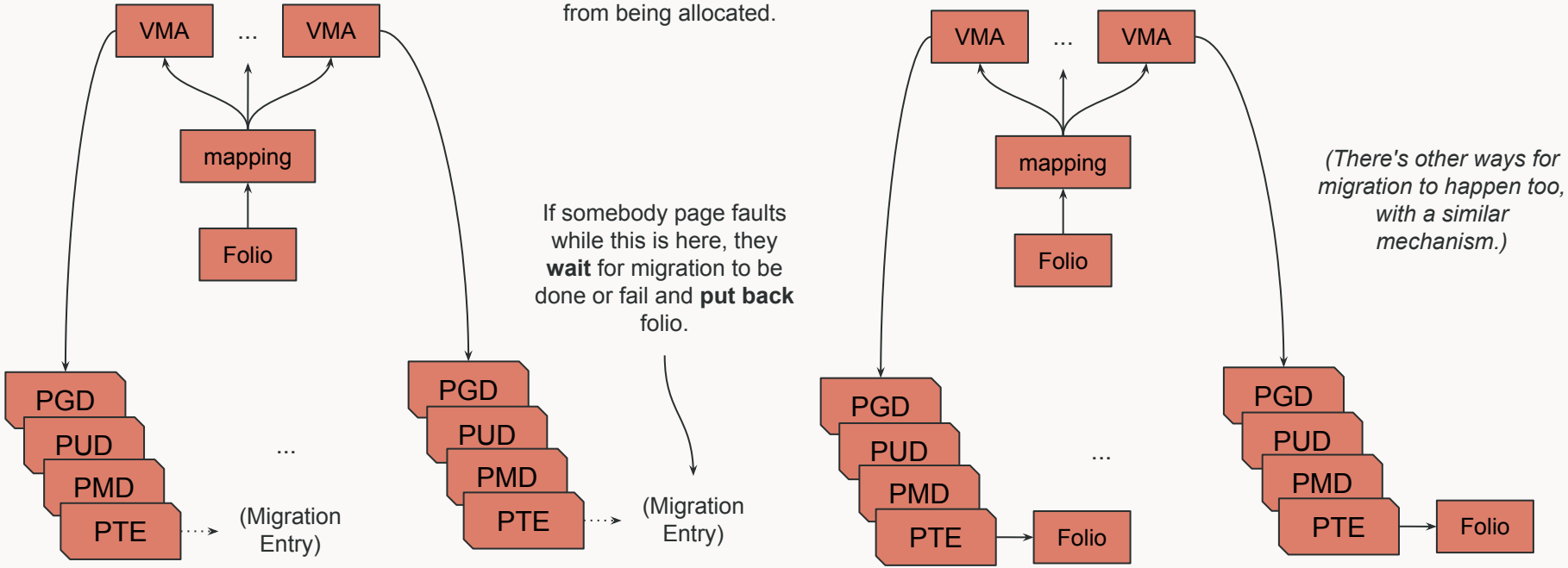
Copyright © 2025, Oracle and/or its affiliates

Compaction/Migration 2

1. Isolate folios by removing them from the LRU, then use the reverse mapping to find all mapped PTEs + set **migration entries.**

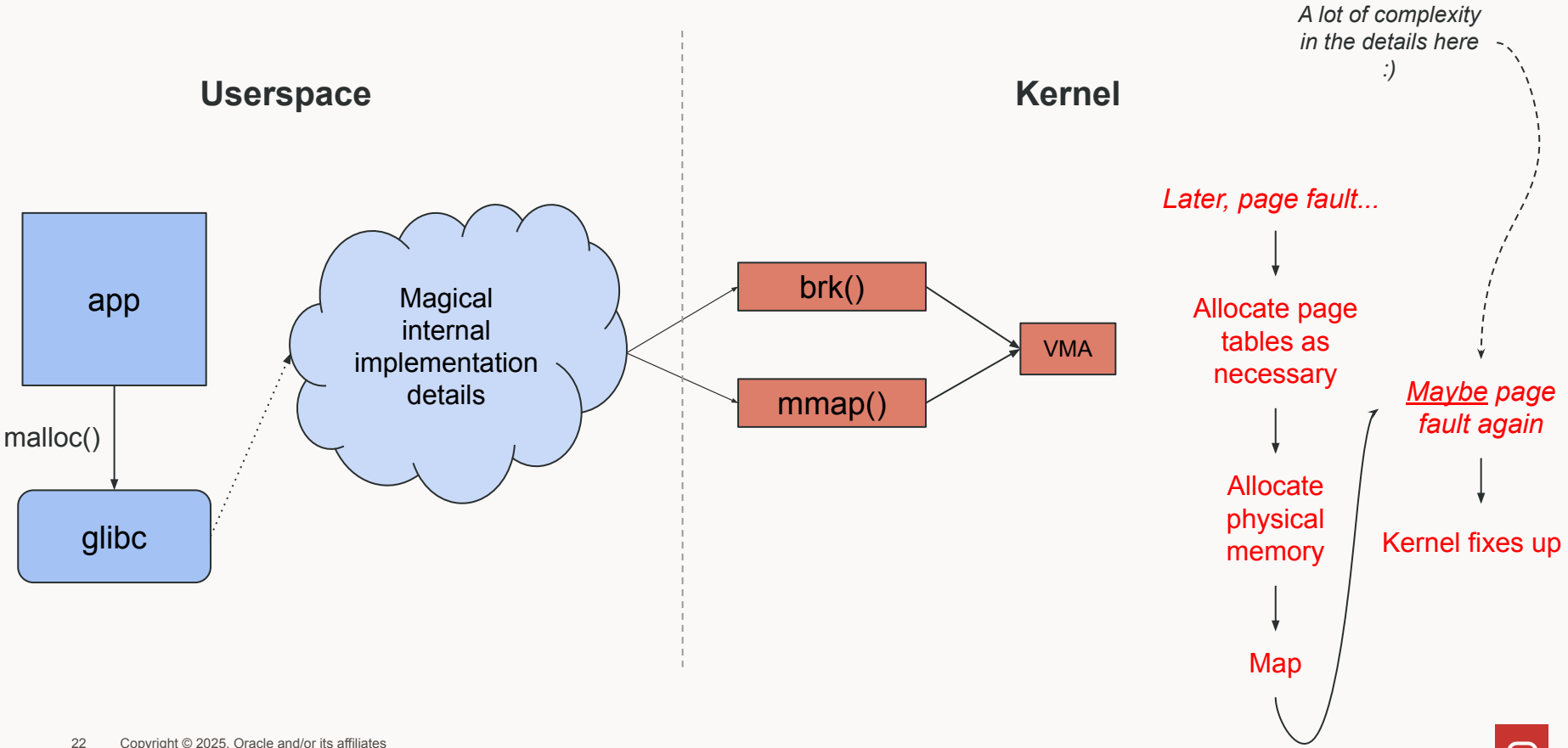
2. Find a suitable destination for each previously mapped folio & isolate those free pages from being allocated.

3. Copy each source folio to the destination, then replace the migration entry with a PTE entry pointing at the now-moved folio, freeing the original.



Copyright © 2025, Oracle and/or its affiliates

Where Does My Memory Come From? Take 4



Copyright © 2025, Oracle and/or its affiliates

Conclusions

- We make allocating memory easy from userland,
- But there's a lot of complexity underneath.
- Managing resources & abstracting complexity is the kernel's job :)
- Memory is a lot more fluid than you might think...
    - It moves around,
    - And can flicker in and out of existence,
    - But when you need it, it's there.
- Virtual memory is *very* powerful.

Copyright © 2025, Oracle and/or its affiliates




ORACLE